- The vector clock idea is very similar to the lamport clock. Both would increment the internal logical clock of a process when an event happens. And in both cases, a process will take the greater value between its own incremented logical clock and the time of the corresponding process in the vector clock received along the message.

  The change in implementation should be relatively easy, as I need to modify the message classes to include the vector clock. And upon receiving messages I also have to update the values of the vector clock kept in the process.

- Five messages sending and receiving steps are needed for a single process to request for resource :
  For a process named as A:
  A receive initial request resource message from system.
  A send request resource message to itself and all other actors; *
  A receive request resource messages from all other actors; *
  A send acknowledgement messages to those actors; *
  A send release resource messages to all other actors; *

  The steps with asterisks marked are scaled linearly with the number of processes. Taking all processes in the system into the consideration, the number of messages sent/received scale at n^2 with n being my number processes. I think it should not be a major problem because each actor/process still only need to hand O(n) messages with increasing processes. In this particular implementation, I can further analyze the throughput of the mailbox and the speed of actor processing the messages to see if there's a bottleneck.

- if one process simply stopped, then the process never get a chance to request resource. As it also has sent request resource messages to all other processes, those processes does not have a mechanism to remove the request message from the dead process. As the algorithm would always try to grant resource to the process with a request message of the lowest logical clock, some processes can get stuck at this point.